

Java Collection Framework - Collection Classes

Objective

To study and implement common Java Collection Framework classes through separate simple programs and demonstrate their important methods.

Classes Included

ArrayList, LinkedList, Vector, Stack, HashSet, LinkedHashSet, TreeSet, PriorityQueue, ArrayDeque, HashMap, LinkedHashMap, TreeMap, and Hashtable.

System Requirements

Hardware: Computer/Laptop, minimum 4 GB RAM, keyboard and mouse. Software: Java Development Kit (JDK), Java compiler, Java Runtime Environment (JRE), and any Java-supported IDE or terminal.

Note

Each program is kept independent and simple. Required classes are imported explicitly; wildcard imports are not used.

Program 1: ArrayList

File Name

ArrayListDemo.java

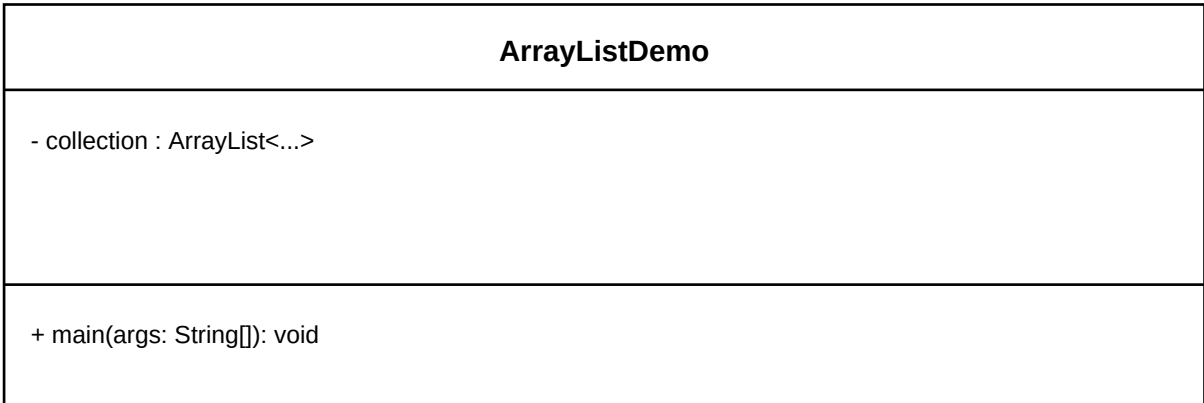
Aim / Objective

To demonstrate basic ArrayList operations such as adding, accessing, updating, searching, removing, and sorting elements.

Algorithm

1. Create an ArrayList.
2. Add elements using add().
3. Access and search elements using get(), contains(), and indexOf().
4. Update an element using set().
5. Remove an element and sort the list.

UML Diagram



ArrayListDemo uses Java ArrayList

Source Code

```
import java.util.ArrayList;

public class ArrayListDemo {
    public static void main(String[] args) {
        ArrayList<String> names = new ArrayList<>();

        names.add("Rahul");           // add() adds an element
        names.add("Priya");
        names.add(1, "Arjun");        // add(index, value) inserts at a position

        System.out.println("ArrayList: " + names);
        System.out.println("Element at index 1: " + names.get(1)); // get() reads an element
        System.out.println("Contains Priya: " + names.contains("Priya")); // contains() checks
        System.out.println("Size: " + names.size()); // size() gives number of elements
        System.out.println("Index of Priya: " + names.indexOf("Priya")); // indexOf() finds position

        names.set(0, "Kiran");        // set() replaces an element
        names.remove("Arjun");        // remove(Object) removes a value
        names.sort(null);             // sort() sorts the list

        System.out.println("Updated ArrayList: " + names);
    }
}
```

Compilation and Execution Commands

Compile: `javac ArrayListDemo.java`

Execute: `java ArrayListDemo`

Output Screen

TERMINAL

```
ArrayList: [Rahul, Arjun, Priya]
Element at index 1: Arjun
Contains Priya: true
Size: 3
Index of Priya: 2
Updated ArrayList: [Kiran, Priya]
```

Result

Thus, the ArrayList program was implemented successfully and the important ArrayList operations were demonstrated.

Program 2: LinkedList

File Name

LinkedListDemo.java

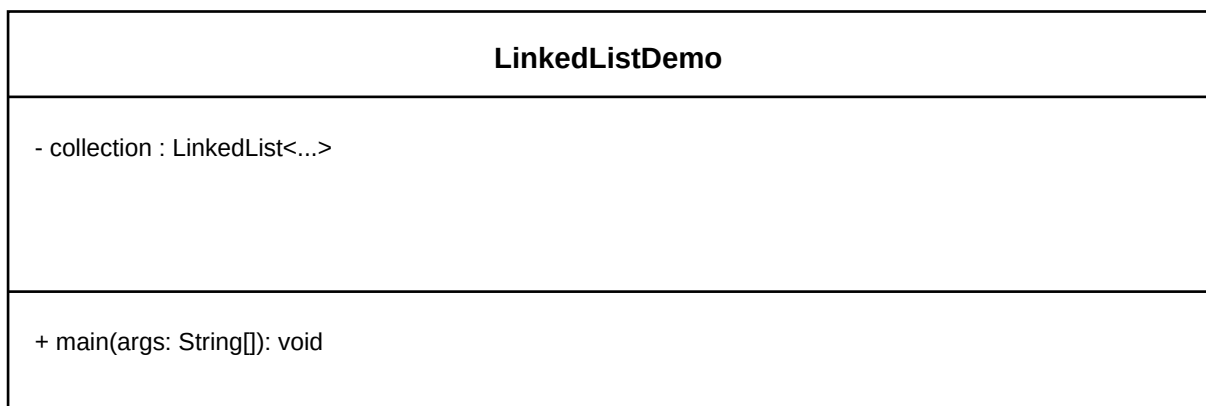
Aim / Objective

To demonstrate insertion, access, queue operations, and removal using LinkedList.

Algorithm

1. Create a LinkedList.
2. Add elements at different positions.
3. Access the first, last, and indexed elements.
4. Use offer(), peek(), and poll().
5. Remove elements from both ends.

UML Diagram



LinkedListDemo uses Java LinkedList

Source Code

```
import java.util.LinkedList;

public class LinkedListDemo {
    public static void main(String[] args) {
        LinkedList<String> names = new LinkedList<>();

        names.add("Rahul");           // add() adds an element
        names.add("Priya");
        names.addFirst("Arjun");       // addFirst() adds at the beginning
        names.addLast("Kiran");        // addLast() adds at the end

        System.out.println("LinkedList: " + names);
        System.out.println("First: " + names.getFirst()); // getFirst() returns first element
        System.out.println("Last: " + names.getLast());   // getLast() returns last element
        System.out.println("Element at index 1: " + names.get(1)); // get() reads by index

        names.offer("Anu");            // offer() adds an element to the queue
        System.out.println("After offer: " + names);
        System.out.println("Peek: " + names.peek());      // peek() reads the first element
        System.out.println("Poll: " + names.poll());      // poll() removes the first element

        names.removeFirst();           // removeFirst() removes the first element
        names.removeLast();            // removeLast() removes the last element

        System.out.println("Final LinkedList: " + names);
    }
}
```

Compilation and Execution Commands

Compile: `javac LinkedListDemo.java`

Execute: `java LinkedListDemo`

Output Screen

TERMINAL

```
LinkedList: [Arjun, Rahul, Priya, Kiran]
First: Arjun
Last: Kiran
Element at index 1: Rahul
After offer: [Arjun, Rahul, Priya, Kiran, Anu]
Peek: Arjun
Poll: Arjun
Final LinkedList: [Priya, Kiran]
```

Result

Thus, the LinkedList program was implemented successfully and the important LinkedList operations were demonstrated.

Program 3: Vector

File Name

VectorDemo.java

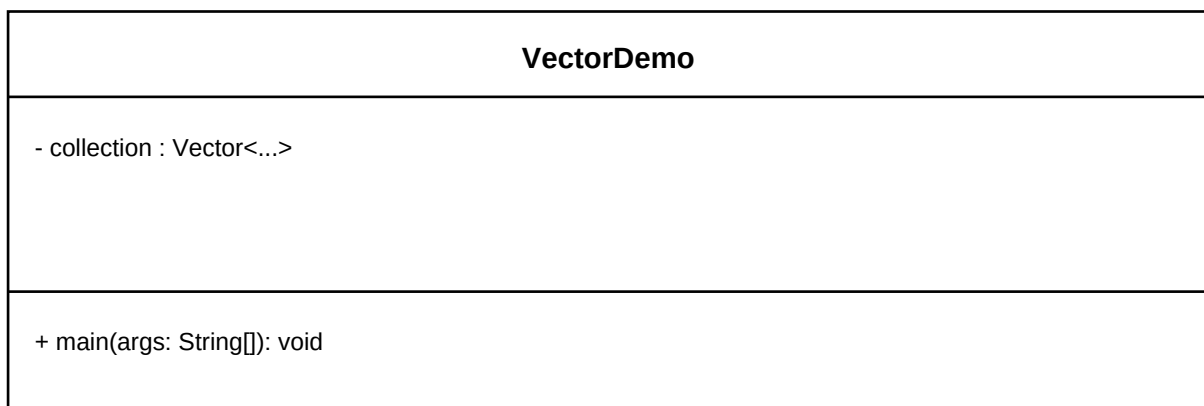
Aim / Objective

To demonstrate common Vector operations including adding, accessing, searching, updating, and removing elements.

Algorithm

1. Create a Vector.
2. Add elements using add() and addElement().
3. Access, search, and find the size and capacity.
4. Update an element using set().
5. Remove elements using remove() and removeElement().

UML Diagram



VectorDemo uses Java Vector

Source Code

```
import java.util.Vector;

public class VectorDemo {
    public static void main(String[] args) {
        Vector<String> names = new Vector<>();

        names.add("Rahul");           // add() adds an element
        names.addElement("Priya");     // addElement() adds an element

        System.out.println("Vector: " + names);
        System.out.println("Element at index 0: " + names.get(0)); // get() reads an element
        System.out.println("Contains Priya: " + names.contains("Priya")); // contains() checks
        System.out.println("Size: " + names.size()); // size() gives number of elements
        System.out.println("Capacity: " + names.capacity()); // capacity() gives current capacity

        names.set(0, "Arjun");         // set() replaces an element
        names.remove("Priya");          // remove() removes a value
        names.removeElement("Arjun");   // removeElement() removes a value

        System.out.println("Final Vector: " + names);
    }
}
```

Compilation and Execution Commands

Compile: `javac VectorDemo.java`

Execute: `java VectorDemo`

Output Screen

TERMINAL

```
Vector: [Rahul, Priya]
Element at index 0: Rahul
Contains Priya: true
Size: 2
Capacity: 10
Final Vector: []
```

Result

Thus, the Vector program was implemented successfully and the important Vector operations were demonstrated.

Program 4: Stack

File Name

StackDemo.java

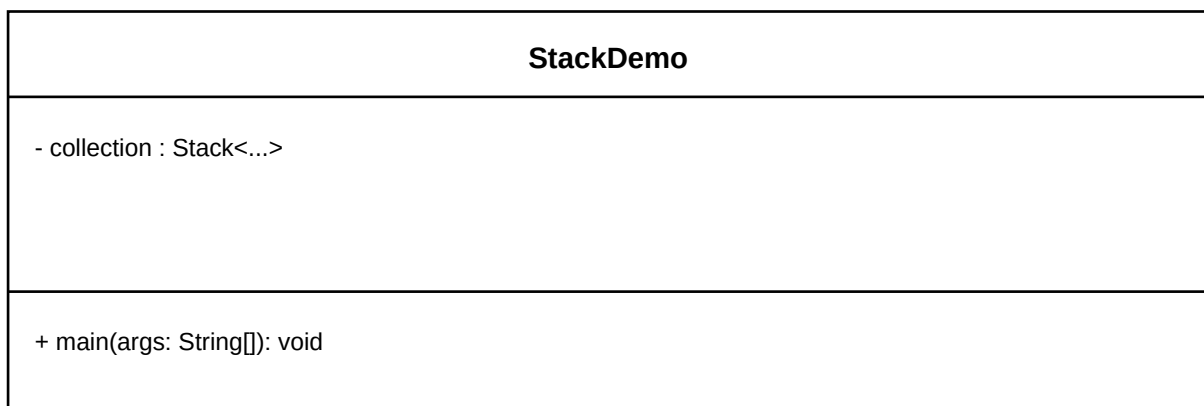
Aim / Objective

To demonstrate stack operations such as push, peek, search, empty, and pop.

Algorithm

1. Create a Stack.
2. Insert elements using push().
3. Access the top using peek() and search for an element.
4. Check whether the stack is empty.
5. Remove the top element using pop().

UML Diagram



StackDemo uses Java Stack

Source Code

```
import java.util.Stack;

public class StackDemo {
    public static void main(String[] args) {
        Stack<Integer> stack = new Stack<>();

        stack.push(10);           // push() adds an element to the top
        stack.push(20);
        stack.push(30);

        System.out.println("Stack: " + stack);
        System.out.println("Top: " + stack.peek()); // peek() reads the top element
        System.out.println("Search 20: " + stack.search(20)); // search() gives position from top
        System.out.println("Is Empty: " + stack.empty()); // empty() checks the stack

        System.out.println("Popped: " + stack.pop()); // pop() removes the top element
        System.out.println("Final Stack: " + stack);
    }
}
```

Compilation and Execution Commands

Compile: `javac StackDemo.java`

Execute: `java StackDemo`

Output Screen

TERMINAL

```
Stack: [10, 20, 30]
Top: 30
Search 20: 2
Is Empty: false
Popped: 30
Final Stack: [10, 20]
```

Result

Thus, the Stack program was implemented successfully and the important Stack operations were demonstrated.

Program 5: HashSet

File Name

HashSetDemo.java

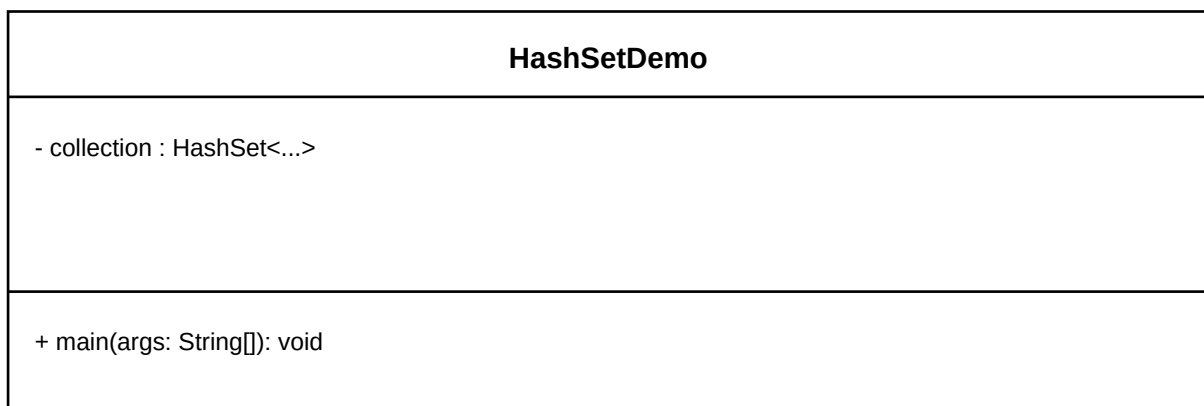
Aim / Objective

To demonstrate HashSet operations and the handling of duplicate elements.

Algorithm

1. Create a HashSet.
2. Add elements and observe that duplicates are ignored.
3. Check an element using contains() and find the size.
4. Remove an element using remove().
5. Clear the set and check isEmpty().

UML Diagram



HashSetDemo uses Java HashSet

Source Code

```
import java.util.HashSet;

public class HashSetDemo {
    public static void main(String[] args) {
        HashSet<Integer> numbers = new HashSet<>();

        numbers.add(10);           // add() adds an element
        numbers.add(20);
        numbers.add(30);
        numbers.add(20);           // duplicate is ignored

        System.out.println("HashSet: " + numbers);
        System.out.println("Contains 20: " + numbers.contains(20)); // contains() checks
        System.out.println("Size: " + numbers.size()); // size() gives number of elements

        numbers.remove(10);        // remove() removes an element
        System.out.println("After remove: " + numbers);

        numbers.clear();           // clear() removes all elements
        System.out.println("Is Empty: " + numbers.isEmpty()); // isEmpty() checks
    }
}
```

Compilation and Execution Commands

Compile: `javac HashSetDemo.java`

Execute: `java HashSetDemo`

Output Screen

TERMINAL

```
HashSet: [20, 10, 30]
Contains 20: true
Size: 3
After remove: [20, 30]
Is Empty: true
```

Result

Thus, the HashSet program was implemented successfully and the important HashSet operations were demonstrated.

Program 6: LinkedHashSet

File Name

LinkedHashSetDemo.java

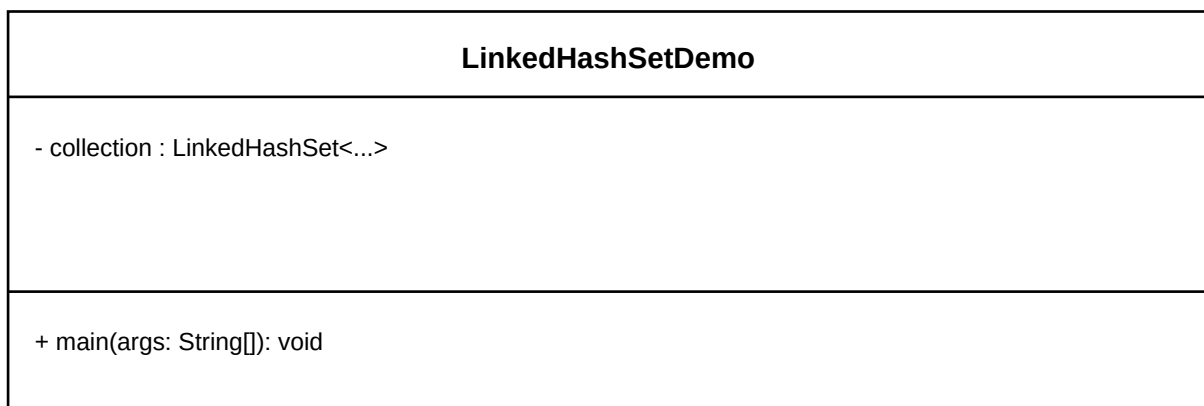
Aim / Objective

To demonstrate LinkedHashSet operations while maintaining insertion order.

Algorithm

1. Create a LinkedHashSet.
2. Add elements and observe insertion order.
3. Check an element using contains() and find the size.
4. Remove an element using remove().
5. Clear the set and check isEmpty().

UML Diagram



LinkedHashSetDemo uses Java LinkedHashSet

Source Code

```
import java.util.LinkedHashSet;

public class LinkedHashSetDemo {
    public static void main(String[] args) {
        LinkedHashSet<String> names = new LinkedHashSet<>();

        names.add("Rahul");           // add() adds an element
        names.add("Priya");
        names.add("Arjun");
        names.add("Priya");           // duplicate is ignored

        System.out.println("LinkedHashSet: " + names);
        System.out.println("Contains Arjun: " + names.contains("Arjun")); // contains() checks
        System.out.println("Size: " + names.size()); // size() gives number of elements

        names.remove("Priya");        // remove() removes an element
        System.out.println("After remove: " + names);

        names.clear();                // clear() removes all elements
        System.out.println("Is Empty: " + names.isEmpty()); // isEmpty() checks
    }
}
```

Compilation and Execution Commands

Compile: `javac LinkedHashSetDemo.java`

Execute: `java LinkedHashSetDemo`

Output Screen

TERMINAL

```
LinkedHashSet: [Rahul, Priya, Arjun]
Contains Arjun: true
Size: 3
After remove: [Rahul, Arjun]
Is Empty: true
```

Result

Thus, the LinkedHashSet program was implemented successfully and the important LinkedHashSet operations were demonstrated.

Program 7: TreeSet

File Name

TreeSetDemo.java

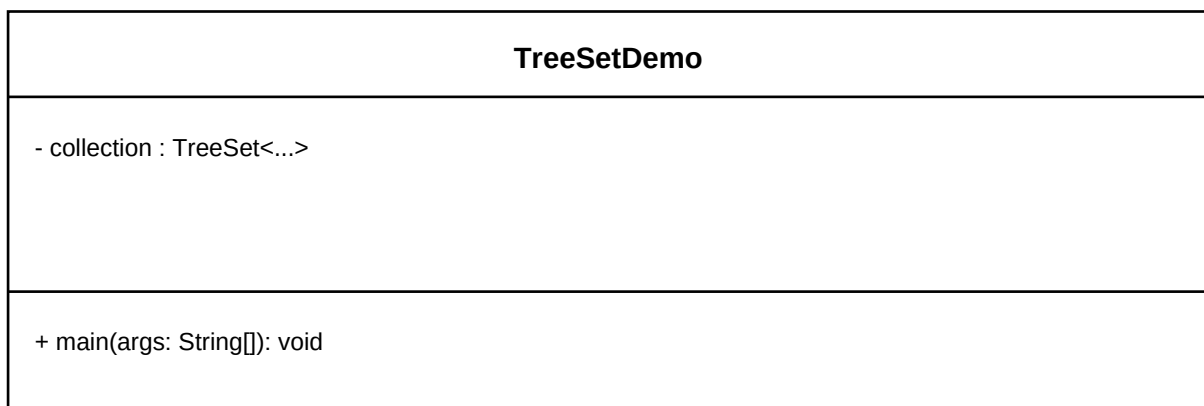
Aim / Objective

To demonstrate sorted-set operations and navigation methods using TreeSet.

Algorithm

1. Create a TreeSet.
2. Add integer elements.
3. Display the first, last, higher, lower, ceiling, and floor elements.
4. Remove the smallest and largest elements using pollFirst() and pollLast().
5. Display the final sorted set.

UML Diagram



TreeSetDemo uses Java TreeSet

Source Code

```
import java.util.TreeSet;

public class TreeSetDemo {
    public static void main(String[] args) {
        TreeSet<Integer> numbers = new TreeSet<>();

        numbers.add(40);           // add() adds an element
        numbers.add(10);
        numbers.add(30);
        numbers.add(20);

        System.out.println("TreeSet: " + numbers);
        System.out.println("First: " + numbers.first()); // first() gives smallest element
        System.out.println("Last: " + numbers.last());  // last() gives largest element
        System.out.println("Higher than 20: " + numbers.higher(20)); // next greater
        System.out.println("Lower than 20: " + numbers.lower(20));  // next smaller
        System.out.println("Ceiling of 25: " + numbers.ceiling(25)); // >= value
        System.out.println("Floor of 25: " + numbers.floor(25));    // <= value

        System.out.println("Poll First: " + numbers.pollFirst()); // removes smallest
        System.out.println("Poll Last: " + numbers.pollLast());  // removes largest
        System.out.println("Final TreeSet: " + numbers);
    }
}
```

Compilation and Execution Commands

Compile: `javac TreeSetDemo.java`

Execute: `java TreeSetDemo`

Output Screen



```
TERMINAL

TreeSet: [10, 20, 30, 40]
First: 10
Last: 40
Higher than 20: 30
Lower than 20: 10
Ceiling of 25: 30
Floor of 25: 20
Poll First: 10
Poll Last: 40
Final TreeSet: [20, 30]
```

Result

Thus, the TreeSet program was implemented successfully and the important TreeSet operations were demonstrated.

Program 8: PriorityQueue

File Name

PriorityQueueDemo.java

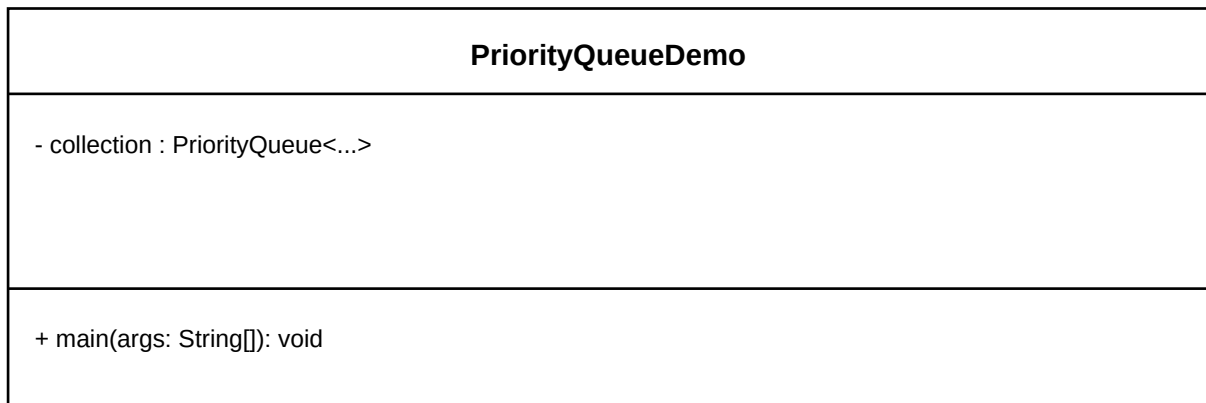
Aim / Objective

To demonstrate priority queue operations such as add, offer, peek, poll, and remove.

Algorithm

1. Create a PriorityQueue.
2. Add elements using add() and offer().
3. Access the highest-priority element using peek().
4. Check the queue using contains() and size().
5. Remove elements using poll() and remove().

UML Diagram



PriorityQueueDemo uses Java PriorityQueue

Source Code

```
import java.util.PriorityQueue;

public class PriorityQueueDemo {
    public static void main(String[] args) {
        PriorityQueue<Integer> queue = new PriorityQueue<>();

        queue.add(30);                // add() adds an element
        queue.offer(10);              // offer() adds an element
        queue.offer(20);

        System.out.println("PriorityQueue: " + queue);
        System.out.println("Peek: " + queue.peek()); // peek() reads highest-priority element
        System.out.println("Contains 20: " + queue.contains(20)); // contains() checks
        System.out.println("Size: " + queue.size()); // size() gives number of elements

        System.out.println("Poll: " + queue.poll()); // poll() removes highest-priority element
        queue.remove(Integer.valueOf(30));           // remove(Object) removes a value

        System.out.println("Final PriorityQueue: " + queue);
    }
}
```

Compilation and Execution Commands

Compile: `javac PriorityQueueDemo.java`

Execute: `java PriorityQueueDemo`

Output Screen

TERMINAL

```
PriorityQueue: [10, 30, 20]
Peek: 10
Contains 20: true
Size: 3
Poll: 10
Final PriorityQueue: [20]
```

Result

Thus, the PriorityQueue program was implemented successfully and the important PriorityQueue operations were demonstrated.

Program 9: ArrayDeque

File Name

ArrayDequeDemo.java

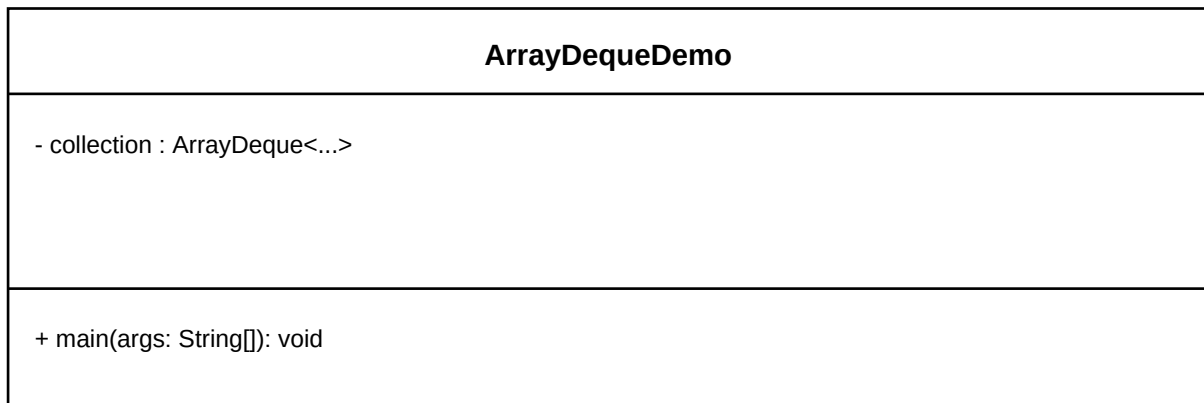
Aim / Objective

To demonstrate double-ended queue operations using ArrayDeque.

Algorithm

1. Create an ArrayDeque.
2. Insert elements at both ends using addFirst(), addLast(), offerFirst(), and offerLast().
3. Read the first and last elements using peekFirst() and peekLast().
4. Remove elements from both ends using pollFirst() and pollLast().
5. Display the final deque.

UML Diagram



ArrayDequeDemo uses Java ArrayDeque

Source Code

```
import java.util.ArrayDeque;

public class ArrayDequeDemo {
    public static void main(String[] args) {
        ArrayDeque<String> deque = new ArrayDeque<>();

        deque.addFirst("B");           // addFirst() adds at the front
        deque.addLast("C");            // addLast() adds at the rear
        deque.offerFirst("A");         // offerFirst() adds at the front
        deque.offerLast("D");          // offerLast() adds at the rear

        System.out.println("ArrayDeque: " + deque);
        System.out.println("First: " + deque.peekFirst()); // peekFirst() reads front
        System.out.println("Last: " + deque.peekLast());   // peekLast() reads rear

        System.out.println("Poll First: " + deque.pollFirst()); // removes front
        System.out.println("Poll Last: " + deque.pollLast());   // removes rear

        System.out.println("Final ArrayDeque: " + deque);
    }
}
```

Compilation and Execution Commands

Compile: `javac ArrayDequeDemo.java`

Execute: `java ArrayDequeDemo`

Output Screen

TERMINAL

```
ArrayDeque: [A, B, C, D]
First: A
Last: D
Poll First: A
Poll Last: D
Final ArrayDeque: [B, C]
```

Result

Thus, the ArrayDeque program was implemented successfully and the important ArrayDeque operations were demonstrated.

Program 10: HashMap

File Name

HashMapDemo.java

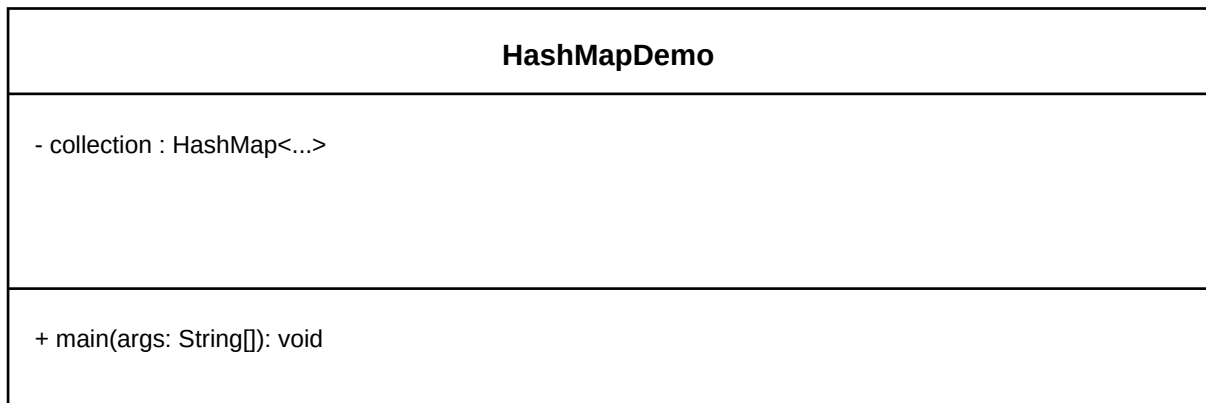
Aim / Objective

To demonstrate key-value operations using HashMap.

Algorithm

1. Create a HashMap with integer keys and string values.
2. Add key-value pairs using put().
3. Access, search, and display keys, values, and entries.
4. Display the size and a default value using getOrDefault().
5. Remove a mapping using remove().

UML Diagram



HashMapDemo uses Java HashMap

Source Code

```
import java.util.HashMap;

public class HashMapDemo {
    public static void main(String[] args) {
        HashMap<Integer, String> students = new HashMap<>();

        students.put(101, "Rahul");          // put() adds a key-value pair
        students.put(102, "Priya");
        students.put(103, "Arjun");

        System.out.println("HashMap: " + students);
        System.out.println("Student 102: " + students.get(102)); // get() returns a value
        System.out.println("Has key 101: " + students.containsKey(101)); // checks key
        System.out.println("Has value Rahul: " + students.containsValue("Rahul")); // checks value
        System.out.println("Keys: " + students.keySet());          // keySet() returns keys
        System.out.println("Values: " + students.values());         // values() returns values
        System.out.println("Entries: " + students.entrySet());     // entrySet() returns pairs
        System.out.println("Size: " + students.size());             // size() gives number of pairs
        System.out.println("Default: " + students.getOrDefault(104, "Not Found")); // default value

        students.remove(103);                                       // remove() removes by key
        System.out.println("Final HashMap: " + students);
    }
}
```

Compilation and Execution Commands

Compile: `javac HashMapDemo.java`

Execute: `java HashMapDemo`

Output Screen

TERMINAL

```
HashMap: {101=Rahul, 102=Priya, 103=Arjun}
Student 102: Priya
Has key 101: true
Has value Rahul: true
Keys: [101, 102, 103]
Values: [Rahul, Priya, Arjun]
Entries: [101=Rahul, 102=Priya, 103=Arjun]
Size: 3
Default: Not Found
Final HashMap: {101=Rahul, 102=Priya}
```

Result

Thus, the HashMap program was implemented successfully and the important HashMap operations were demonstrated.

Program 11: LinkedHashMap

File Name

LinkedHashMapDemo.java

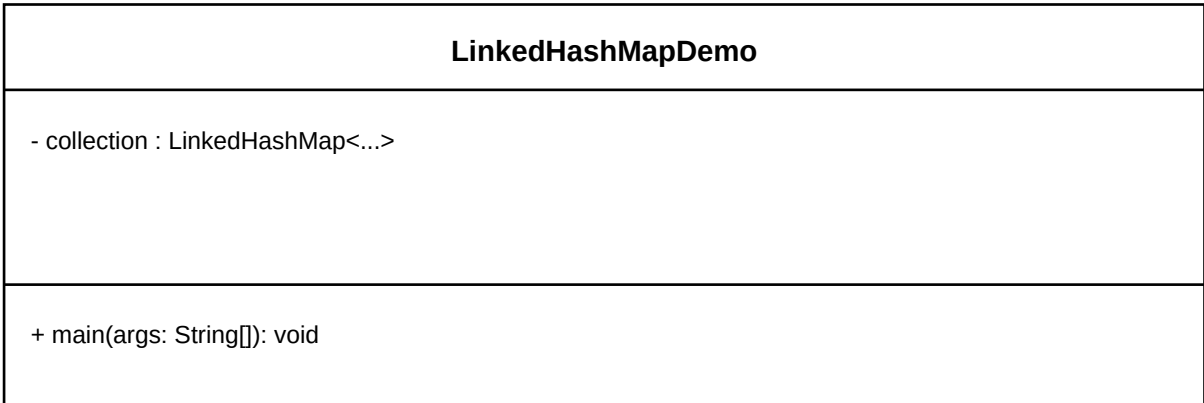
Aim / Objective

To demonstrate key-value operations using LinkedHashMap while maintaining insertion order.

Algorithm

- 1. Create a LinkedHashMap.
- 2. Add key-value pairs using put().
- 3. Access a value and check a key using get() and containsKey().
- 4. Display keys, values, and entries.
- 5. Remove a mapping using remove().

UML Diagram



LinkedHashMapDemo uses Java LinkedHashMap

Source Code

```
import java.util.LinkedHashMap;

public class LinkedHashMapDemo {
    public static void main(String[] args) {
        LinkedHashMap<Integer, String> students = new LinkedHashMap<>();

        students.put(101, "Rahul");          // put() adds a key-value pair
        students.put(102, "Priya");
        students.put(103, "Arjun");

        System.out.println("LinkedHashMap: " + students);
        System.out.println("Student 102: " + students.get(102)); // get() returns a value
        System.out.println("Has key 101: " + students.containsKey(101)); // checks key
        System.out.println("Keys: " + students.keySet());           // keySet() returns keys
        System.out.println("Values: " + students.values());          // values() returns values
        System.out.println("Entries: " + students.entrySet());       // entrySet() returns pairs

        students.remove(102);                                         // remove() removes by key
        System.out.println("Final LinkedHashMap: " + students);
    }
}
```

Compilation and Execution Commands

Compile: `javac LinkedHashMapDemo.java`

Execute: `java LinkedHashMapDemo`

Output Screen

TERMINAL

```
LinkedHashMap: {101=Rahul, 102=Priya, 103=Arjun}
Student 102: Priya
Has key 101: true
Keys: [101, 102, 103]
Values: [Rahul, Priya, Arjun]
Entries: [101=Rahul, 102=Priya, 103=Arjun]
Final LinkedHashMap: {101=Rahul, 103=Arjun}
```

Result

Thus, the LinkedHashMap program was implemented successfully and the important LinkedHashMap operations were demonstrated.

Program 12: TreeMap

File Name

TreeMapDemo.java

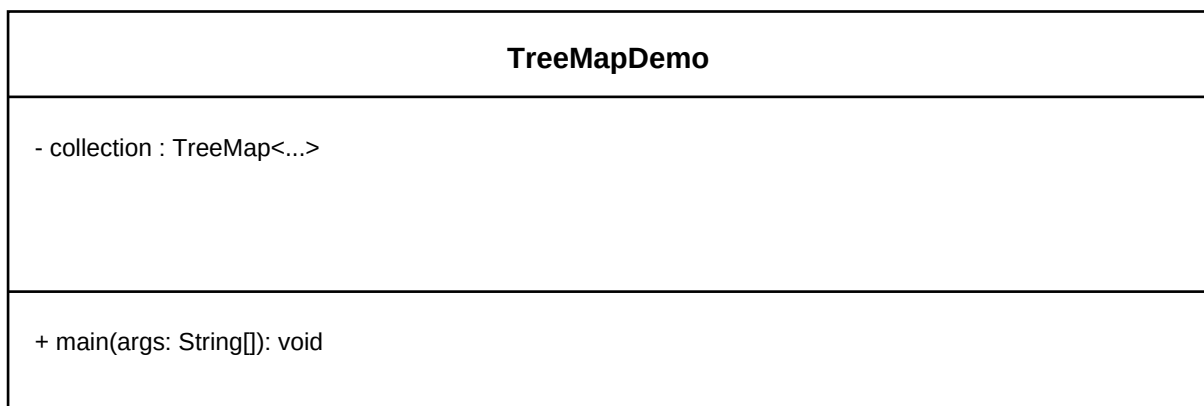
Aim / Objective

To demonstrate sorted key-value operations and navigation methods using TreeMap.

Algorithm

1. Create a TreeMap.
2. Add key-value pairs using put().
3. Access a value using get().
4. Find first, last, higher, lower, ceiling, and floor keys.
5. Remove a mapping and display the final sorted map.

UML Diagram



TreeMapDemo uses Java TreeMap

Source Code

```
import java.util.TreeMap;

public class TreeMapDemo {
    public static void main(String[] args) {
        TreeMap<Integer, String> students = new TreeMap<>();

        students.put(103, "Arjun");          // put() adds a key-value pair
        students.put(101, "Rahul");
        students.put(102, "Priya");
        students.put(104, "Kiran");

        System.out.println("TreeMap: " + students);
        System.out.println("Student 102: " + students.get(102)); // get() returns a value
        System.out.println("First Key: " + students.firstKey()); // smallest key
        System.out.println("Last Key: " + students.lastKey());   // largest key
        System.out.println("Higher Key: " + students.higherKey(102)); // next greater
        System.out.println("Lower Key: " + students.lowerKey(102)); // next smaller
        System.out.println("Ceiling Key: " + students.ceilingKey(102)); // >= key
        System.out.println("Floor Key: " + students.floorKey(102)); // <= key

        students.remove(104);                // remove() removes by key
        System.out.println("Final TreeMap: " + students);
    }
}
```

Compilation and Execution Commands

Compile: `javac TreeMapDemo.java`

Execute: `java TreeMapDemo`

Output Screen

TERMINAL

```
TreeMap: {101=Rahul, 102=Priya, 103=Arjun, 104=Kiran}
Student 102: Priya
First Key: 101
Last Key: 104
Higher Key: 103
Lower Key: 101
Ceiling Key: 102
Floor Key: 102
Final TreeMap: {101=Rahul, 102=Priya, 103=Arjun}
```

Result

Thus, the TreeMap program was implemented successfully and the important TreeMap operations were demonstrated.

Program 13: Hashtable

File Name

HashtableDemo.java

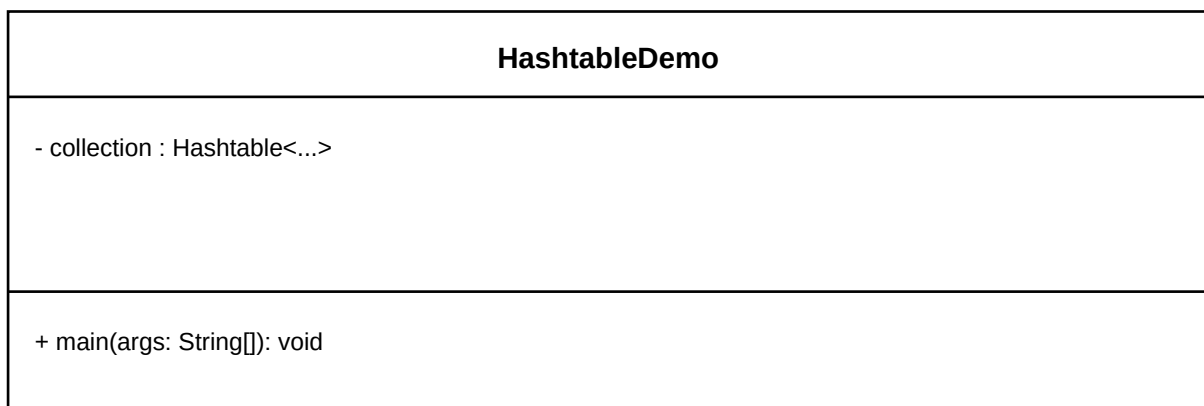
Aim / Objective

To demonstrate key-value operations using Hashtable.

Algorithm

1. Create a Hashtable.
2. Add key-value pairs using put().
3. Access and search keys and values.
4. Display keys, elements, and size.
5. Remove a mapping and check whether the table is empty.

UML Diagram



HashtableDemo uses Java Hashtable

Source Code

```
import java.util.Hashtable;

public class HashtableDemo {
    public static void main(String[] args) {
        Hashtable<Integer, String> students = new Hashtable<>();

        students.put(101, "Rahul");           // put() adds a key-value pair
        students.put(102, "Priya");
        students.put(103, "Arjun");

        System.out.println("Hashtable: " + students);
        System.out.println("Student 102: " + students.get(102)); // get() returns a value
        System.out.println("Has key 101: " + students.containsKey(101)); // checks key
        System.out.println("Has value Rahul: " + students.containsValue("Rahul")); // checks value
        System.out.println("Keys: " + students.keys());           // keys() returns an enumeration of keys
        System.out.println("Elements: " + students.elements());   // elements() returns values
        System.out.println("Size: " + students.size());           // size() gives number of pairs

        students.remove(103);                                     // remove() removes by key
        System.out.println("Is Empty: " + students.isEmpty());    // isEmpty() checks the table
        System.out.println("Final Hashtable: " + students);
    }
}
```

Compilation and Execution Commands

Compile: `javac HashtableDemo.java`

Execute: `java HashtableDemo`

Output Screen

TERMINAL

```
Hashtable: {103=Arjun, 102=Priya, 101=Rahul}
Student 102: Priya
Has key 101: true
Has value Rahul: true
Keys: java.util.Hashtable$Enumerator@5cad8086
Elements: java.util.Hashtable$Enumerator@6e0be858
Size: 3
Is Empty: false
Final Hashtable: {102=Priya, 101=Rahul}
```

Result

Thus, the Hashtable program was implemented successfully and the important Hashtable operations were demonstrated.